ASSIGNMENT- 9.1

Name : K.shiva

H.T.NO: 2303A51246

Batch : 18

Problem 1:

Consider the following Python function:

def find_max(numbers):

return max(numbers)

Task:

• Write documentation for the function in all three formats:

(a) Docstring

(b) Inline comments

(c) Google-style documentation

• Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.

• Recommend which documentation style is most effective for a mathematical utilities library and justify your answer

Promt :

Write documentation for the function in three formats

and compare them. Also recommend the best style.

(a) Docstring Style

(b) Inline Comment Style

This function takes a list of numbers

It uses Python's built-in max() function

to return the largest number

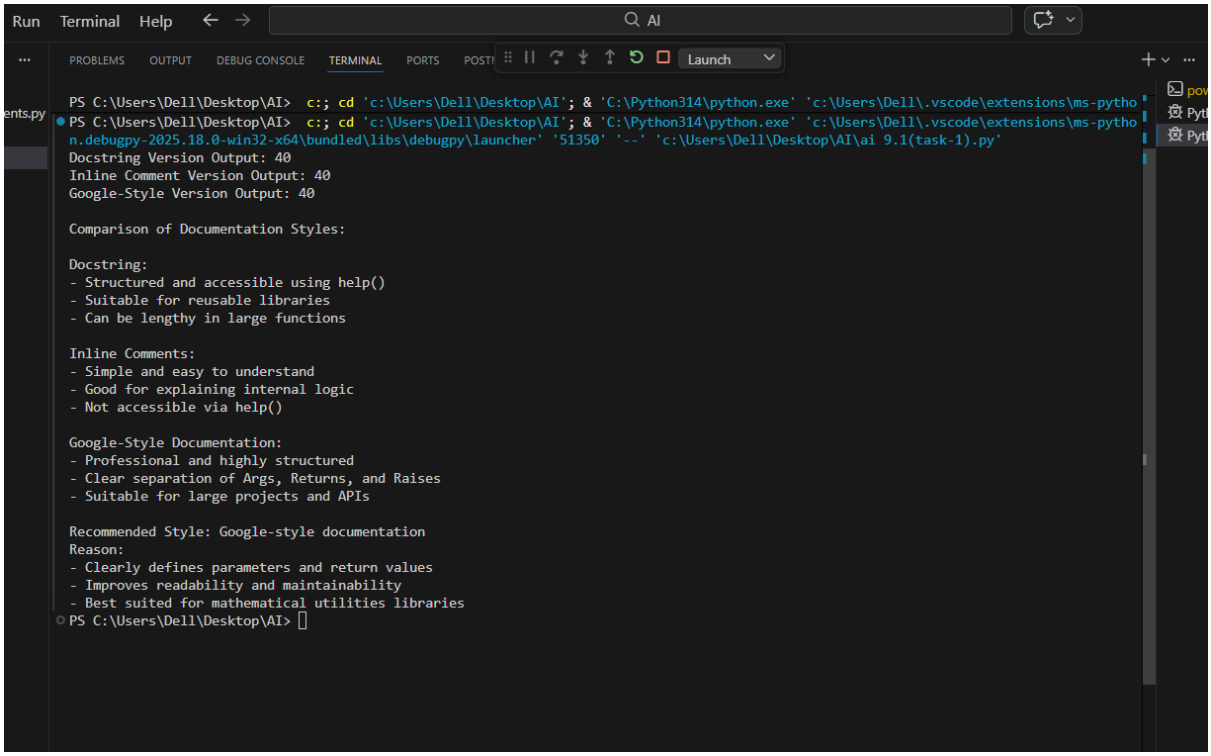
(c) Google-Style Documentation

INPUT:

```python
# Write documentation for the function in three formats
# and compare them. Also recommend the best style.


# (a) Docstring Style

def find_max_docstring(numbers):
    """
    Returns the maximum value from a list of numbers.

    Parameters:
        numbers (list): A list of numeric values.

    Returns:
        int or float: The largest value in the list.

    Raises:
        ValueError: If the list is empty.
    """
    return max(numbers)


# (b) Inline Comment Style

def find_max_inline(numbers):
    # This function takes a list of numbers

    # It uses Python's built-in max() function
    # to return the largest number

    return max(numbers)


#  (c) Google-Style Documentation

def find_max_google(numbers):
    """Returns the maximum value from a list of numbers.

    Args:
        numbers (list[int | float]): A list of numeric values.

```

```python
def find_max_google(numbers):

    Raises:
        ValueError: If the list is empty.
    """
    return max(numbers)


# ----------- Example Execution -----------

numbers = [10, 25, 5, 40, 15]

print("Docstring Version Output:", find_max_docstring(numbers))
print("Inline Comment Version Output:", find_max_inline(numbers))
print("Google-Style Version Output:", find_max_google(numbers))


# ----------- Comparison & Recommendation Output -----------

print("\nComparison of Documentation Styles:\n")

print("Docstring:")
print("- Structured and accessible using help()")
print("- Suitable for reusable libraries")
print("- Can be lengthy in large functions\n")

print("Inline Comments:")
print("- Simple and easy to understand")
print("- Good for explaining internal logic")
print("- Not accessible via help()\n")

print("Google-Style Documentation:")
print("- Professional and highly structured")
print("- Clear separation of Args, Returns, and Raises")
print("- Suitable for large projects and APIs\n")

print("Recommended Style: Google-style documentation")
print("Reason:")
print("- Clearly defines parameters and return values")
print("- Improves readability and maintainability")
print("- Best suited for mathematical utilities libraries")
```

OUTPUT:



EXPLANATION:

Docstring Style – Explanation

Docstrings are written inside triple quotes below a function and provide a clear description of what the function does. They explain inputs, outputs, and possible errors in a readable format. Docstrings can be accessed using help() and are useful for documentation tools. They are good for reusable functions and structured code.

Inline Comment Style – Explanation

Inline comments are written using # inside the function body. They explain the logic of specific lines of code and improve readability. However, they are not structured documentation and cannot be accessed automatically. They are mainly useful for explaining complex steps in the code.

Google-Style Documentation – Explanation

Google-style documentation is a well-structured docstring format that clearly separates sections like Args, Returns, and Raises. It improves clarity and consistency in larger projects. This style is widely used in professional development. It is best suited for libraries and production-level code

Problem 2:

Consider the following Python function:

```
def login(user, password, credentials):
return credentials.get(user) == password
```

Task:

1. Write documentation in all three formats.
2. Critically compare the approaches.
3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

PROMT:

Write documentation for the login function using standard Python docstring format.

#Rewrite the same login function documentation using inline comments.

#Rewrite the login function using Google-style documentation format.

#Create example credentials data and test all three versions of the function.

#Compare docstring, inline comments, and Google-style documentation

in terms of structure, clarity, and maintainability.

Discuss advantages and disadvantages of each documentation style.

#Recommend the most suitable documentation style

for onboarding new developers to a project.

Justify the recommendation with clear reasoning.

INPUT:

Run Terminal Help AI

ai(6.3 task -2).py | ai(6.3 task-3).py | ai 9.3(task-1) | Launch | ai 9.1(task-1).py | ai 9.1(task-2).py

ai 9.1(task-2).py > ...

```python
# Write documentation for this login function in different formats
# and compare them. Finally, suggest which style is best for
# new developers joining a project.


# Docstring style documentation
def login_docstring(user, password, credentials):
    """
    This function checks whether the given username and password
    match the stored credentials.

    Parameters:
        user (str): Username of the person logging in.
        password (str): Password entered by the user.
        credentials (dict): Dictionary containing username-password pairs.

    Returns:
        bool: True if login is successful, otherwise False.
    """
    return credentials.get(user) == password


# Same function explained using inline comments
def login_inline(user, password, credentials):
    # Take username and password as input
    # Look up the stored password from the credentials dictionary
    # Compare stored password with entered password
    return credentials.get(user) == password


# Google-style documentation format
def login_google(user, password, credentials):
    """Validates user login using stored credentials.

    Args:
        user (str): Username provided for login.
        password (str): Password entered by the user.
        credentials (dict[str, str]): Dictionary storing valid credentials.

    Returns:
        bool: True if authentication is successful, False otherwise.
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
making it easier for new developers to understand the function quickly.
PS C:\Users\Dell\Desktop\AI>
```

Terminal Help AI

ai(6.3 task -2).py | ai(6.3 task-3).py | ai 9.3(task-1) | Launch | -3).py | ai 9.1(task-1).py | ai 9.1(ta

ai 9.1(task-2).py > ...

```python
def login_google(user, password, credentials):
    return credentials.get(user) == password


# Example data to test the functions
credentials_data = {
    "admin": "1234",
    "user1": "pass",
    "john": "abcd"
}

print("Docstring Version:", login_docstring("admin", "1234", credentials_data))
print("Inline Version:", login_inline("user1", "wrong", credentials_data))
print("Google Version:", login_google("john", "abcd", credentials_data))


# Comparing the three documentation approaches

print("\nComparison of Documentation Styles:\n")

print("Docstring:")
print("Structured and visible using help().")
print("Good for reusable functions but may become lengthy.\n")

print("Inline Comments:")
print("Easy to understand and helpful for simple logic.")
print("Not structured and cannot be accessed using help().\n")

print("Google-Style:")
print("Professional and clearly separates inputs and outputs.")
print("Best suited for team projects and larger applications.\n")


# Recommendation for onboarding new developers

print("Recommended Style: Google-style documentation.")
print("Reason: It clearly explains parameters and return values,")
print("making it easier for new developers to understand the function quickly.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
making it easier for new developers to understand the function quickly.
PS C:\Users\Dell\Desktop\AI>
```

OUTPUT:

```
- Best suited for mathematical utilities libraries
PS C:\Users\Dell\Desktop\AI>  c:; cd 'c:\Users\Dell\Desktop\AI'; & 'C:\Python314\python.exe' 'c:\Users\Dell\.vscode\extensions\ms-pytho
n.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56666' '--' 'c:\Users\Dell\Desktop\AI\ai 9.1(task-2).py'
Docstring Version: True
Inline Version: False
Google Version: True

Comparison of Documentation Styles:

Docstring:
Structured and visible using help().
Good for reusable functions but may become lengthy.

Inline Comments:
Good for reusable functions but may become lengthy.

Inline Comments:

Inline Comments:
Inline Comments:
Easy to understand and helpful for simple logic.
Not structured and cannot be accessed using help().


Google-Style:
Professional and clearly separates inputs and outputs.
Best suited for team projects and larger applications.

Recommended Style: Google-style documentation.
Recommended Style: Google-style documentation.
Reason: It clearly explains parameters and return values,
making it easier for new developers to understand the function quickly.
PS C:\Users\Dell\Desktop\AI>
```

EXPLANATION:

Docstrings are written inside triple quotes below a function. They explain the purpose, parameters, and return values clearly. They can be accessed using help() and are useful for documentation tools. This style is good for reusable and well-structured code.

Inline Comment Style – Explanation

Inline comments use # to explain specific lines of code. They help understand the internal logic step-by-step. However, they are not structured and cannot be accessed as formal documentation. They are best for explaining simple or complex logic inside the function.

Google-Style Documentation – Explanation

Google-style documentation is a structured docstring format with clear sections like Args and Returns. It improves clarity and consistency in team projects. It is widely used in industry and suitable for larger applications.

Recommendation

Google-style documentation is best for new developers because it clearly explains inputs and outputs in a structured way, making the function easy to understand quickly

Problem 3: Calculator (Automatic Documentation Generation)

Task: Design a Python module named calculator.py and demonstrate automatic documentation generation.

Instructions:

1. Create a Python module calculator.py that includes the following functions, each written with appropriate docstrings:

o add(a, b) – returns the sum of two numbers

o subtract(a, b) – returns the difference of two numbers

o multiply(a, b) – returns the product of two numbers

o divide(a, b) – returns the quotient of two numbers

2. Display the module documentation in the terminal using Python's documentation tools.

3. Generate and export the module documentation in HTML format using the pydoc utility, and open the generated HTML file in a web browser to verify the output.

PROMT:

Prompt: Create basic arithmetic functions (add, subtract, multiply, divide).

Prompt: Add simple docstrings to each function for automatic documentation generation.

Prompt: Use help() to display documentation of each function in the terminal.

Prompt: Execute each function with sample input values.

Prompt: Display the output of all arithmetic operations clearly.

INPUT :

```python
# Prompt: Create basic arithmetic functions with proper docstrings.

def add(a, b):
    """Return the sum of two numbers."""
    return a + b


def subtract(a, b):
    """Return the difference of two numbers."""
    return a - b


def multiply(a, b):
    """Return the product of two numbers."""
    return a * b


def divide(a, b):
    """Return the quotient of two numbers."""
    if b == 0:
        return "Cannot divide by zero"
    return a / b


# Prompt: Demonstrate automatic documentation generation using help().

print("=== Calculator Documentation ===\n")
help(add)
help(subtract)
help(multiply)
help(divide)


# Prompt: Demonstrate function execution with sample values.

print("\n=== Function Outputs ===")
print("Add:", add(10, 5))
print("Subtract:", subtract(10, 5))
print("Multiply:", multiply(10, 5))
print("Divide:", divide(10, 5))
```

```
Divide: 2.0
PS C:\Users\Dell\Desktop\AI>
```

OUTPUT:

```
Help on function divide in module __main__:

divide(a, b)
    Return the quotient of two numbers.
divide(a, b)
    Return the quotient of two numbers.

    Return the quotient of two numbers.



=== Function Outputs ===
=== Function Outputs ===
Add: 15
Subtract: 5
Multiply: 50
Divide: 2.0
PS C:\Users\Dell\Desktop\AI>
```

EXPLANATION:

Docstring Explanation

In this program, each arithmetic function contains a simple docstring that briefly explains its purpose. The docstring is written inside triple quotes directly below the function definition. When the help() function is called, Python automatically displays this documentation. This demonstrates how docstrings support automatic documentation generation.

Importance of Using Docstrings

Docstrings make functions self-explanatory and improve readability. They allow developers to understand what a function does without reading its implementation. Using help() is especially useful in larger projects, libraries, and team environments. This approach ensures clean, professional, and maintainable code

Problem 4: Conversion Utilities Module

Task:

1. Write a module named conversion.py with functions:

o decimal_to_binary(n)

o binary_to_decimal(b)

o decimal_to_hexadecimal(n)

2. Use Copilot for auto-generating docstrings.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a

Browser

PROMPTS:

Create a Python module named conversion.py with number conversion functions.

Implement decimal_to_binary(n) to convert a decimal number to binary.

Implement binary_to_decimal(b) to convert a binary string to decimal.

Implement decimal_to_hexadecimal(n) to convert a decimal number to hexadecimal.

Add proper docstrings to all functions (as if generated by Copilot).

Use help() to display function documentation in the terminal.

Demonstrate the conversion functions using sample inputs.

Generate module documentation in HTML format using the pydoc utility.

Open the generated HTML file in a web browser to verify the documentation.

INPUT:

```python
# Prompt: Create a module named conversion.py with required conversion functions.
# Prompt: Add proper docstrings (as auto-generated by Copilot) for documentation.

def decimal_to_binary(n):
    """
    Convert a decimal integer to its binary representation.

    Args:
        n (int): Decimal number.

    Returns:
        str: Binary string equivalent of the number.
    """
    return bin(n)[2:]


def binary_to_decimal(b):
    """
    Convert a binary string to its decimal representation.

    Args:
        b (str): Binary number as a string.

    Returns:
        int: Decimal equivalent of the binary number.
    """
    return int(b, 2)


def decimal_to_hexadecimal(n):
    """
    Convert a decimal integer to its hexadecimal representation.

    Args:
        n (int): Decimal number.

    Returns:
        str: Hexadecimal string equivalent of the number.
    """
    return hex(n)[2:]
```

OUTPUT:

```
28
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

Decimal to Binary (10): 1010
Binary to Decimal (1010): 10
Binary to Decimal (1010): 10
Decimal to Hexadecimal (255): ff
PS C:\Users\Dell\Desktop\AI>
```

Explanation:

Each function in the module contains a structured docstring that explains its purpose, input arguments, and return values. These docstrings follow a clear format similar to professional documentation standards. When help() is used, Python automatically reads and displays these docstrings in the terminal. This makes the module self-documented and easy to understand.

Automatic Documentation Generation

The help() function demonstrates how Python can generate documentation directly from docstrings. Additionally, using the pydoc -w conversion command creates an HTML file containing the module documentation. This shows how properly written docstrings support both terminal-based and web-based documentation generation, making the code more professional and maintainable.

Problem 5 – Course Management Module

Task:

1. Create a module course.py with functions:

o add_course(course_id, name, credits)

o remove_course(course_id)

o get_course(course_id)

2. Add docstrings with Copilot.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a

browser.

PROMPTS:

COURSE MANAGEMENT MODULE

This module should allows adding, removing, and retrieving

course details. It also demonstrates documentation

generation using help().

In-memory course storage

INPUT:

```
k-2).py   ai 9.3(task-3).py   ai 9.1(task-1).py   ai 9.1   Launch   9.1(task-4.py   ai
ai 9.1 (task-5).py > ...
1
2   # COURSE MANAGEMENT MODULE
3   # This module should allows adding, removing, and retrieving
4   # course details. It also demonstrates documentation
5   # generation using help().
6
\Users\Dell\Desktop\AI\ai 9.1(task-2).py   torage
8   courses = {}
9
10
11  def add_course(course_id, name, credits):
12      """
13      Add a new course to the course management system.
14
15      Args:
16          course_id (str): Unique identifier for the course.
17          name (str): Name of the course.
18          credits (int): Credit value assigned to the course.
19
20      Returns:
21          str: Success or error message based on operation result.
22      """
23      if course_id in courses:
24          return "Course ID already exists."
25
26      if credits <= 0:
27          return "Credits must be a positive number."
28
29      courses[course_id] = {
30          "name": name,
31          "credits": credits
32      }
33      return "Course added successfully."
34
35
36  def remove_course(course_id):
37      """
38      Remove an existing course from the system.
39
40      Args:
41          course_id (str): Unique identifier of the course.
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE
```

Run Terminal Help AI

k-2).py | ai 9.3(task-3).py | ai 9.1(task-1).py | ai 9.1(| Launch | 9.1(task-4.py

ai 9.1 (task-5).py > ...

```python
def remove_course(course_id):
    Returns:
        str: Success or error message.
    """
    if course_id in courses:
        del courses[course_id]
        return "Course removed successfully."
    return "Course not found."


def get_course(course_id):
    """
    Retrieve course details using course ID.

    Args:
        course_id (str): Unique identifier of the course.

    Returns:
        dict or str: Course details if found, otherwise error message.
    """
    return courses.get(course_id, "Course not found.")



if __name__ == "__main__":

    print("=========================================")
    print("      COURSE MANAGEMENT SYSTEM")
    print("=========================================")

    while True:

        print("\nPlease choose an option below:")
        print("1. Add a New Course")
        print("2. Remove an Existing Course")
        print("3. View Course Details")
        print("4. Show Function Documentation")
        print("5. Exit")

        choice = input("Enter your choice (1-5): ")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

5. Exit

Terminal Help AI

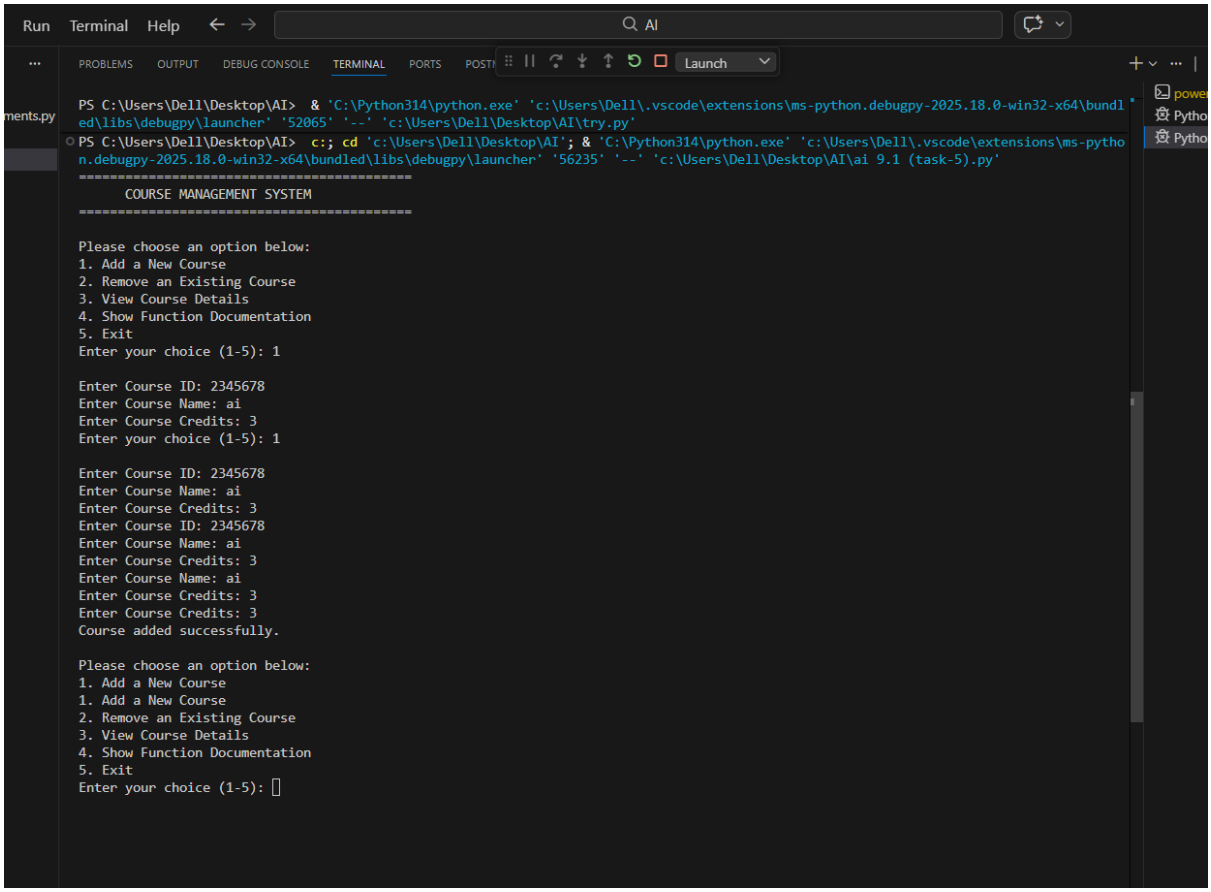
k-2).py | ai 9.3(task-3).py | ai 9.1(task-1).py | ai 9.1 | Launch | 9.1(task-4.py | ai 9.1 (task-5).py

ai 9.1 (task-5).py > ...

```python
        # GET COURSE DETAILS

        elif choice == "3":

            course_id = input("\nEnter Course ID to view details: ").strip()
            result = get_course(course_id)

            if isinstance(result, dict):
                print("\nCourse Details:")
                print("Course ID :", course_id)
                print("Name      :", result["name"])
                print("Credits   :", result["credits"])
            else:
                print(result)


        # SHOW DOCUMENTATION USING help()

        elif choice == "4":

            print("\n=== FUNCTION DOCUMENTATION ===\n")
            help(add_course)
            help(remove_course)
            help(get_course)

            print("\nTo export documentation in HTML format, run this in terminal:")
            print("pydoc -w course")

            print("\nThen open the generated file using:")
            print("Windows : start course.html")
            print("Mac     : open course.html")



        elif choice == "5":
            print("\nExiting Course Management System...")
            break

        else:
            print("Invalid choice. Please select between 1 and 5.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

OUTPUT:



Explanation:

This program is a simple course management system that allows users to add, remove, and view course details through a menu-driven interface. It stores course information in a dictionary and performs basic validation like checking duplicate IDs and valid credits. Each function includes proper docstrings to support documentation generation. The documentation can be viewed using help() or exported to HTML format using the pydoc command.